

第 10 讲: 反向传播

神经网络如何计算参数梯度

@ Frano

● FranSpace

本讲摘要 / Why it matters

反向传播 (Backpropagation, BP) 是神经网络训练中的核心算法。前向传播负责从输入得到预测结果, 损失函数负责衡量预测误差, 而反向传播负责从损失出发, 利用链式法则计算损失对所有可训练参数的梯度。需要特别注意的是:

反向传播本身并不直接更新参数, 它只负责计算梯度; 真正决定参数如何更新的是优化器。理解反向传播, 是理解梯度下降、Adam、L-BFGS 以及 PINN 训练机制的关键。

10.1 为什么需要反向传播?

在前面的内容中, 我们已经知道, 神经网络训练的目标是让损失函数尽可能小。假设网络中所有参数记为 θ , 损失函数为 $L(\theta)$, 训练过程可以写成

$$\theta^* = \arg \min_{\theta} L(\theta).$$

为了让损失函数下降, 我们需要知道每个参数应该往哪个方向调整。这就需要计算损失函数对参数的导数, 也就是梯度:

$$\nabla_{\theta} L.$$

如果只有一个参数, 这个问题很简单; 但真实神经网络可能有成千上万甚至上亿个参数。如果逐个手工推导每个参数的导数, 几乎不可能完成。反向传播的作用, 就是用一种系统化、高效的方式, 自动计算损失函数对所有参数的梯度。

分析 / Insight

反向传播解决的是“如何高效计算所有参数梯度”的问题。它不是优化器，也不是某种特殊的参数更新公式，而是一个基于链式法则的梯度计算方法。

10.2 训练流程中的反向传播

完整的神经网络训练流程通常包括以下步骤：

1. 前向传播：输入数据经过网络得到预测结果；
2. 计算损失：用损失函数衡量预测结果和目标之间的差距；
3. 反向传播：从损失出发，计算损失对每个参数的梯度；
4. 参数更新：优化器根据梯度修改参数。

反向传播：从损失出发，沿计算图反向计算每个参数的梯度

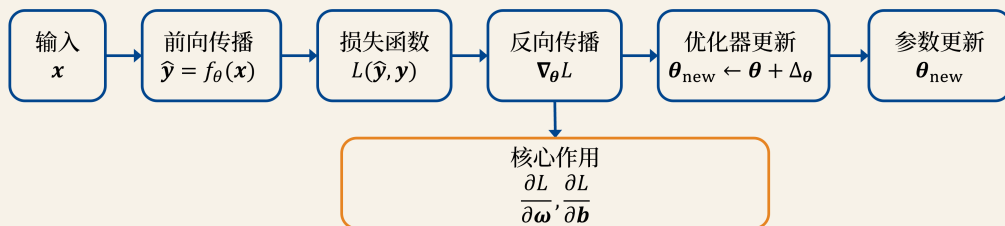


图 1: 反向传播在神经网络训练流程中的位置

这张图的重点是区分两个概念：反向传播负责计算梯度，优化器负责使用梯度更新参数。例如最简单的梯度下降更新公式是

$$\theta_{k+1} = \theta_k - \eta \nabla_{\theta} L(\theta_k).$$

其中 $\nabla_{\theta} L$ 是反向传播计算出来的， η 是学习率，参数如何更新由优化器规则决定。

10.3 梯度到底表示什么

梯度可以理解为损失函数对参数的敏感性。如果某个参数稍微改变一点，损失函数变化很大，说明损失对这个参数很敏感；如果某个参数改变一点，损失几乎不变，说明这个参数当前对损失影响较小。

对于一个参数 w 来说，损失对它的导数为

$$\frac{\partial L}{\partial w}.$$

如果这个导数为正，说明增加 w 会使损失增大，因此为了降低损失，应该减小 w ；如果这个导数为负，说明增加 w 会使损失减小，因此为了降低损失，应该增大 w 。

因此，梯度不仅告诉我们“当前错了多少”，更重要的是告诉我们“参数应该往哪个方向改”。

Example / 例子

假设某个参数 w 当前的梯度为

$$\frac{\partial L}{\partial w} = 5.$$

这说明如果增大 w ，损失会增大。因此，梯度下降会让 w 朝相反方向更新：

$$w_{\text{new}} = w - \eta \cdot 5.$$

如果梯度为 -5 ，则参数会向增大的方向更新。

10.4 链式法则是反向传播的数学基础

反向传播的核心数学工具是链式法则。神经网络本质上是很多函数的复合。比如输入先经过线性层，再经过激活函数，再经过下一层，最后得到损失。可以抽象成

$$x \rightarrow z \rightarrow \hat{y} \rightarrow L.$$

如果想计算 L 对 x 的导数，就不能只看最后一步，而要把中间所有依赖关系都串起来：

$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial z} \frac{\partial z}{\partial x}.$$

这就是链式法则。反向传播就是把这个思想推广到多层网络和大量参数上。

备注 / Note

链式法则的直观含义是：如果 x 影响 z ， z 影响 $\hat{y}$ ， $\hat{y}$ 又影响 L ，那么 x 对 L 的影响就等于这条影响路径上各个局部影响的乘积。

10.5 单神经元中的反向传播

先看一个最简单的单神经元例子。设

$$z = wx + b,$$

并假设输出为

$$\hat{y} = z.$$

损失函数取为

$$L = \frac{1}{2}(\hat{y} - y)^2.$$

现在我们想计算损失对权重 w 的梯度。根据链式法则：

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial z} \frac{\partial z}{\partial w}.$$

逐项计算：

$$\frac{\partial L}{\partial \hat{y}} = \hat{y} - y,$$

$$\frac{\partial \hat{y}}{\partial z} = 1,$$

$$\frac{\partial z}{\partial w} = x.$$

因此

$$\frac{\partial L}{\partial w} = (\hat{y} - y)x.$$

同理，损失对偏置 b 的梯度为

$$\frac{\partial L}{\partial b} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial z} \frac{\partial z}{\partial b} = (\hat{y} - y).$$

单神经元中的链式法则：局部导数相乘得到参数梯度

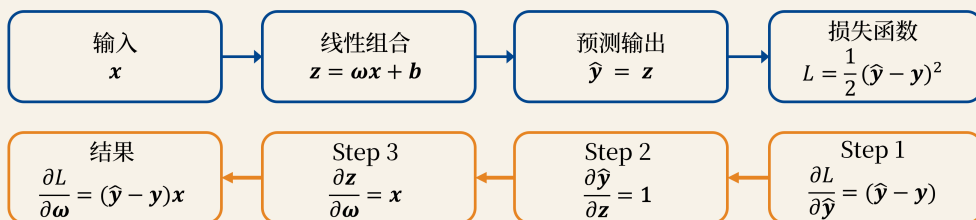


图 2: 单神经元中利用链式法则计算参数梯度

这个例子虽然简单，但已经包含了反向传播的本质：先前向计算中间量，再从损失出发反向计算每一步的局部导数，最后把它们相乘得到参数梯度。

10.6 计算图的作用

为了让计算机自动完成反向传播，深度学习框架会构建计算图 (Computational Graph)。计算图记录了每个变量如何由前面的变量计算而来。

例如：

$$x, w, b \rightarrow z = wx + b \rightarrow \hat{y} \rightarrow L.$$

在前向传播时，计算图记录每一步运算以及必要的中间量；在反向传播时，框架沿着计算图反向应用链式法则，把梯度逐步传回每个参数。

这也是为什么在 PyTorch 中，如果某个变量没有参与计算图，或者中途被 `detach()` 截断，那么它就无法得到正确梯度。

10.7 计算图的作用

要理解 PyTorch 中的反向传播，必须先理解计算图 (Computational Graph)。计算图可以理解为：记录每一个变量是如何由其他变量一步一步计算出来的关系图。

例如，考虑一个最简单的单神经元：

$$\begin{aligned} z &= wx + b, \\ \hat{y} &= z, \\ L &= \frac{1}{2}(\hat{y} - y)^2. \end{aligned}$$

从前向传播的角度看，计算过程是：

$$x, w, b \rightarrow z \rightarrow \hat{y} \rightarrow L.$$

这条链路说明： z 由 x, w, b 计算得到， $\hat{y}$ 由 z 得到， L 又由 $\hat{y}$ 和 y 得到。计算图的作用，就是把这些依赖关系保存下来。

• 10.7.1 前向传播时计算图记录了什么

在前向传播中，程序不只是算出最终的 loss，还会记录中间变量之间的关系。例如：

$$z = wx + b$$

这一步中，计算图会记录： z 依赖于 w 、 x 和 b 。

再比如：

$$L = \frac{1}{2}(\hat{y} - y)^2$$

这一步中，计算图会记录： L 依赖于 $\hat{y}$ 和 y 。

这些记录不是为了前向传播本身，而是为了后面的反向传播。因为反向传播需要知道损失 L 是如何一步一步由参数 w, b 计算出来的。只有知道这条路径，才能使用链式法则把梯度从 L 传回到 w 和 b 。

分析 / Insight

计算图可以理解成反向传播的“路线图”。前向传播负责把路线走一遍并记录路径，反向传播则沿着这条路径反向计算每个参数对 loss 的影响。

10.7.2 反向传播为什么需要计算图

反向传播要计算的是：

$$\frac{\partial L}{\partial w}, \quad \frac{\partial L}{\partial b}.$$

但是 L 并不是直接由 w 和 b 得到的，而是经过了中间变量：

$$w, b \rightarrow z \rightarrow \hat{y} \rightarrow L.$$

因此，计算 $\frac{\partial L}{\partial w}$ 时，必须沿着依赖关系反向使用链式法则：

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial z} \frac{\partial z}{\partial w}.$$

如果没有计算图，程序就不知道 w 是通过哪几步影响到 L 的，也就无法自动计算这个梯度。

备注 / Note

反向传播不是凭空计算梯度。它依赖前向传播过程中保存下来的中间变量和计算关系。如果这些中间信息被删除或截断，梯度就无法正确传回参数。

10.8 多层网络中的反向传播

对多层神经网络，第 l 层通常写成

$$z^{(l)} = W^{(l)} a^{(l-1)} + b^{(l)},$$

$$a^{(l)} = \sigma(z^{(l)}).$$

前向传播从输入层逐层计算到输出层；反向传播则从损失函数开始，逐层向前传回梯度。对于最后一层，可以先计算

$$\frac{\partial L}{\partial a^{(L)}}.$$

然后根据激活函数导数得到

$$\frac{\partial L}{\partial z^{(L)}}.$$

再继续计算

$$\frac{\partial L}{\partial \mathbf{W}^{(L)}}, \quad \frac{\partial L}{\partial \mathbf{b}^{(L)}}.$$

接着，这个梯度会继续传向上一层，直到所有层的权重和偏置都得到梯度。

从直观上看，多层网络中的反向传播就像“责任分配”：最终损失由输出误差产生，但这个误差来自每一层参数共同作用。反向传播要做的事，就是把最终损失按照计算路径逐层分配回每个参数。

10.9 为什么反向传播是高效的

如果不用反向传播，而是对每个参数单独做数值差分，成本会非常高。假设网络有 M 个参数，对每个参数都要扰动一次并重新计算损失，那么每次训练可能需要 M 次额外前向传播。

反向传播的优势在于，它通过一次前向传播和一次反向传播，就可以同时得到所有参数的梯度。这使得训练大规模神经网络成为可能。

可以粗略理解为：

- 数值差分：每个参数单独试探，成本随参数数量急剧增加；
- 反向传播：利用计算图和链式法则，一次性高效计算所有参数梯度。

分析 / Insight

反向传播之所以重要，不只是因为它能算梯度，更因为它能高效地算梯度。没有反向传播，现代大规模神经网络训练几乎不可行。

10.10 PyTorch 中的反向传播

在 PyTorch 中，反向传播最常见的调用方式是

```
loss.backward().
```

一个标准训练步骤通常是：

1. `optimizer.zero_grad()`：清空上一轮梯度；
2. `pred = model(x)`：前向传播；
3. `loss = loss_fn(pred, y)`：计算损失；
4. `loss.backward()`：反向传播计算梯度；
5. `optimizer.step()`：优化器更新参数。

这里要特别注意，PyTorch 中梯度默认会累积。如果不先调用 `zero_grad()`，新的梯度会加到旧梯度上，导致参数更新不符合预期。

备注 / Note

在 PyTorch 中，`backward()` 只负责把梯度计算并存入参数的 `.grad` 属性中；真正修改参数的是 `optimizer.step()`。

10.11 反向传播和自动微分的关系

反向传播通常可以理解为自动微分 (Automatic Differentiation, AD) 在神经网络训练中的具体应用。自动微分并不是数值差分，也不是单纯符号求导，而是根据计算图精确地应用链式法则。

在 PINN 中，自动微分不仅用于计算损失对网络参数的梯度，还常用于计算网络输出对输入坐标的导数。例如：

$$\frac{\partial u_{\theta}}{\partial X}, \quad \frac{\partial u_{\theta}}{\partial Y}.$$

这些导数用于构造 PDE 残差、应变、应力等物理量。也就是说，在普通监督学习中，自动微分主要服务于参数更新；而在 PINN 中，自动微分还直接参与物理方程的构造。

10.12 PINN 中的反向传播

在 PINN 中，反向传播比普通监督学习更复杂。因为 PINN 的损失函数通常不是单一的数据误差，而是多个物理约束项的组合：

$$L = \lambda_r L_r + \lambda_D L_D + \lambda_N L_N + \lambda_{\text{data}} L_{\text{data}}.$$

其中，PDE 残差损失可能依赖网络输出的一阶、二阶甚至更高阶导数。例如在固体力学 PINN 中，网络预测位移场：

$$\mathbf{u}_{\theta}(\mathbf{X}).$$

通过自动微分得到位移梯度：

$$\nabla \mathbf{u}_{\theta}.$$

再进一步构造变形梯度：

$$\mathbf{F} = \mathbf{I} + \nabla \mathbf{u}_{\theta}.$$

然后根据应变能函数计算应力，再通过应力散度构造平衡方程残差。

这意味着 PINN 的反向传播不仅要处理网络参数，还要处理由自动微分构造出的复杂物理量。因此，PINN 中的计算图通常比普通监督学习更深、更复杂，也更容易出现显存占用高、梯度不稳定或计算图断裂等问题。

PINN中的反向传播: 梯度不仅来自数据误差, 也来自物理残差与边界约束

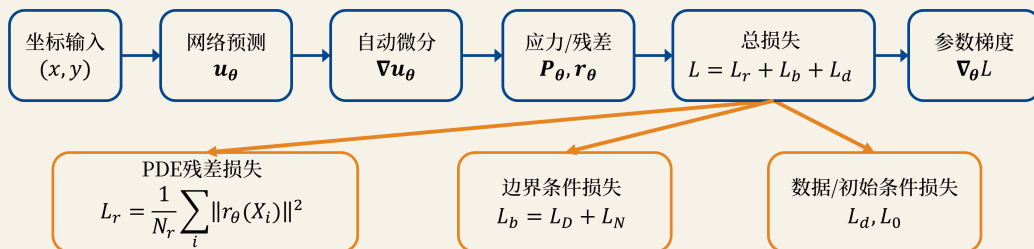


图 3: PINN 中反向传播与物理残差的关系

10.13 反向传播中的常见问题

反向传播虽然由框架自动完成, 但使用时仍然容易出错。常见问题包括:

- 忘记清空梯度, 导致梯度累积;
- 使用 `detach()` 使计算图中断;
- 在不该使用的地方使用 `torch.no_grad()`;
- 中途把 Tensor 转为 NumPy 数组, 导致自动微分失效;
- 对坐标求导时没有设置 `requires_grad=True`;
- PINN 中计算高阶导数时忘记设置 `create_graph=True`;
- loss 中某些项没有真正参与计算图, 导致对应参数收不到梯度。

这些错误有时不会立即报错, 但会导致训练结果异常。例如 loss 看起来在下降, 但 PDE 残差没有真正被优化; 或者边界 loss 数值存在, 但没有正确影响参数更新。

10.14 如何检查反向传播是否正常

在调试神经网络或 PINN 时, 可以从以下几个方面检查反向传播是否正常:

- 检查参数的 `.grad` 是否为 `None`;
- 检查梯度范数是否长期为 0;
- 分别打印每个 loss 项, 确认它们都参与训练;
- 对简单算例做手算或有限差分对比;
- 检查是否有 `detach()`、`no_grad()`、`numpy()` 打断计算图;

- 对 PINN，检查坐标张量是否设置了 `requires_grad=True`。

Example / 例子

如果某个 PINN 程序中 PDE loss 一直不变，可能不是优化器无效，而是 PDE residual 的计算图被截断了。此时应检查 residual 是否由网络输出通过自动微分计算得到，是否中途使用了 `detach()` 或 NumPy 操作。

10.15 反向传播、梯度下降和优化器的区别

到这里需要再次区分三个概念：

- 反向传播：负责计算梯度；
- 梯度下降：是一种最基本的参数更新思想；
- 优化器：根据梯度和自身规则决定参数如何更新。

例如 Adam 会利用历史梯度的一阶矩和二阶矩，L-BFGS 会利用历史参数变化和梯度变化近似曲率信息。它们都需要梯度，而梯度通常由反向传播提供。

因此可以把关系总结为：

反向传播算梯度，优化器用梯度更新参数。

10.16 一页总结

- 反向传播 (Backpropagation, BP) 的核心作用是计算损失函数对所有可训练参数的梯度。
- 反向传播的数学基础是链式法则，它把复合函数的导数分解成多个局部导数的乘积。
- 计算图 (Computational Graph) 记录了前向传播中的依赖关系，反向传播沿计算图反向传递梯度。
- PyTorch 中 `loss.backward()` 负责计算梯度，`optimizer.step()` 负责更新参数。
- 在 PINN 中，反向传播更加复杂，因为 loss 可能依赖网络输出、输出对坐标的导数、应力和 PDE 残差。
- 调试 PINN 时，需要特别注意 `requires_grad=True`、`create_graph=True`、`detach()` 和 `no_grad()` 等计算图问题。
- 反向传播不是优化器。它只负责算梯度，具体如何使用梯度由优化器决定。